

Guião da Apresentação Oral — APF-T

Sistema de Gestão de Coworking · Bruno Pereira (112726) + Rafael Claro (088860) **Duração-alvo:** 6 minutos · **20 slides + demo ao vivo**

Convenção: **[B]** = Bruno fala · **[R]** = Rafael fala · *(itálico)* = nota de palco / o que mostrar no ecrã.
Tempos cumulativos à direita. Falar pausado: ~130 palavras/min.

Bloco 1 — Abertura & Contexto · ~0:45

Slide 1 — Título · ~0:15

[B] «Boa tarde. Somos o Bruno Pereira e o Rafael Claro, e o nosso projeto de Bases de Dados é um **Sistema de Gestão de Coworking** — gestão de espaços de trabalho partilhados, do registo do cliente até ao pagamento.»

Slide 2 — Contexto / O que resolve · ~0:30

[B] «O sistema cobre dois lados. À esquerda, o domínio principal: **clientes**, **planos** — Flex, Fixo e Privado —, **adesões**, e a **reserva** de salas à hora ou de postos ao dia. À direita, as operações transversais: **notificações automáticas**, **lista de espera**, **reservas recorrentes** e **auditoria temporal**.» **[B]** «A stack é **SQL Server** com **C# WinForms** em **.NET 8**, e toda a comunicação com a base de dados é **ADO.NET parametrizado** — sem ORM, e imune a SQL injection por construção.»

Bloco 2 — Modelo de Dados · ~1:15 *(passa para o Rafael)*

Slide 3 — DER · ~0:25

[R] «Passo ao modelo de dados. Este é o nosso **Diagrama Entidade-Relacionamento**. O ponto central é o **recurso** como *supertype*: um recurso **é-uma sala ou é-um posto**. Isto permite-nos tratar reservas de salas e de postos de forma uniforme, com uma só relação.»

Slide 4 — Esquema Relacional · ~0:30

[R] «Traduzido para o esquema relacional, ficam **12 tabelas em BCNF**. Reparem em três pormenores: a **adesao** guarda o `preco_acordado` — um snapshot do preço do plano; a **reserva** é a entidade associativa entre cliente e recurso, com atributos próprios; e o **pagamento** liga-se *ou* a uma adesão *ou* a uma reserva, em **XOR**, e guarda o `preco_servico_snapshot`.» **[R]** «Estes dois snapshots respondem diretamente às correções do professor — já lá vou.»

Bloco 3 — Implementação SQL · ~2:30 *(Rafael e Bruno alternam)*

Slide 5 — SQL Scripts · ~0:15

[B] «A implementação está organizada em **8 scripts por tópico**: DDL, funções, triggers, views, stored procedures, tabelas temporais, índices e o seed de dados. Mostro os destaques.»

Slide 6 — DDL: supertype `recurso` · ~0:20

[B] «No DDL, o *supertype* é feito com **PK igual a FK**: a `sala` e o `posto` partilham a chave do `recurso`, com **CASCADE delete**. Assim uma única FK em `reserva.recurso_id` cobre os dois tipos de recurso.»

Slide 7 — UDF `fn_recurso_disponivel` · ~0:20

[R] «Esta **função** verifica se um recurso está livre num intervalo — a condição de **sobreposição** de horários. É usada em dois sítios: pela **aplicação**, antes de oferecer a reserva, e pelo **trigger**, antes de aceitar o INSERT. A mesma regra, num só lugar.»

Slide 8 — SP `sp_criar_reserva_sala` (concorrência) · ~0:30

[R] «A reserva é criada por *stored procedure*. O detalhe importante é a **concorrência**: o trigger sozinho deixaria passar uma *race condition* — dois INSERTs simultâneos para a mesma sala e dia. Resolvemos com `sp_getapplock` em modo *Exclusive*, dentro da transação, que **serializa** os pedidos concorrentes para o mesmo recurso.»

Slide 9 — View `vw_top_clientes_receita` · ~0:15

[B] «As **views** alimentam os relatórios. Esta faz o ranking de clientes por receita, com `RANK()` — para tratar empates corretamente — e `LEFT JOIN`, para que clientes sem pagamentos apareçam na mesma com receita zero.»

Slide 10 — Trigger T6: snapshot do preço · ~0:30 (*correção #1*)

[B] «Aqui está a **primeira correção do professor** — “qual o preço?” no pagamento. Garantimos o valor cobrado em **três níveis**: a coluna `preco_servico_snapshot`, um **CHECK constraint**, e este **trigger**, que faz *rollback* se o snapshot não bater certo com o valor da reserva. O valor cobrado fica **imutável**, mesmo que o preço do serviço mude depois.»

Slide 11 — Índices · ~0:10

[R] «Em performance: um índice composto faz a deteção de sobreposições passar de *table scan* a *index seek*, e os **filtered indexes** nos pagamentos poupam cerca de metade do armazenamento, ignorando as linhas a NULL.»

Slide 12 — Tabelas Temporais · ~0:10

[R] «E a auditoria é **automática**: com `SYSTEM_VERSIONING`, cada alteração em adesão, reserva e pagamento guarda a versão anterior numa tabela de histórico — sem trigger nem schema manual. Conseguimos reconstituir o estado em qualquer instante passado.»

Bloco 4 — Aplicação & Demo · ~1:15 (*volta ao Bruno*)

Slides 13-17 — UI (passar rápido) · ~0:20

[B] «Do lado da aplicação: um **Dashboard** com KPIs e gráficos, com toggle **light/dark** persistente; a **tab Mapa** das reservas, com timeline horária; os **pagamentos** com geração de **recibo em PDF**; e o ecrã de **estatísticas**, alimentado pelas views que vimos.» (*Avançar pelos screenshots sem parar em cada um.*)

Slide 18 — DEMO ao vivo · ~0:55

[B] «Mas, em vez de slides, vamos mostrar ao vivo.» (*Demo curta e ensaiada* — Bruno conduz, Rafael comenta. *Sequência sugerida:*)

1. [B] *Dashboard* — «O status bar mostra **LIGADO**: estamos a falar mesmo com o SQL Server.»
2. [B] *Reservas* → *criar reserva sobreposta* — «Se tentar reservar uma sala já ocupada...» → [R] «...o trigger rejeita: a regra está na base de dados, não na aplicação.»
3. [B] *Pagamentos* → *Recibo PDF* — «Geramos o recibo com o preço congelado.»
4. [R] (*se houver tempo*) «E aqui o toggle dark e as estatísticas em tempo real.»

Bloco 5 — Fecho · ~0:20

Slide 19 — Em números · ~0:15

[R] «Em resumo: **12 tabelas em BCNF**, **16 triggers**, mais de **18 stored procedures**, **11 views** e **7 funções**. Regras de negócio em três níveis, concorrência segura, auditoria temporal — e as **duas correções do professor resolvidas**: o snapshot do preço e a reserva como entidade associativa.»

Slide 20 — Obrigado · ~0:05

[B] «Obrigado. Estamos disponíveis para perguntas.»

Resumo da divisão

Bloco	Slides	Quem
Abertura & Contexto	1-2	Bruno
Modelo de Dados	3-4	Rafael
SQL — DDL / View / Trigger	5, 6, 9, 10	Bruno
SQL — UDF / SP / Índices / Temporal	7, 8, 11, 12	Rafael
UI + Demo	13-18	Bruno conduz , Rafael comenta
Em números	19	Rafael
Obrigado	20	Bruno

Tempo total estimado: ~6:00 · *Folga:* cortar slide 9 (View) e encurtar a demo se passar do tempo.

Dicas de palco

- **Ensaiar a demo offline primeiro** — ter a app já aberta e ligada antes de começar (não gastar tempo no F5 à frente do júri).
- Ter um **plano B** caso a ligação à BD falhe na demo: screenshots dos slides 13-17 cobrem tudo.
- A **troca de orador** faz-se nas transições de bloco (slides 3, 5, 7...) — combinar uma frase-ponte: *"passo ao Rafael / volto ao Bruno"*.
- Não ler o código linha a linha — apontar **só a ideia** de cada slide.
- Guardar **30s de folga** mental: se o tempo apertar, os slides 9, 11 e 12 podem ser ditos numa frase cada.